LedgerPilot

Security Self-Assessment Report

2026 FinTechathon · WeBank FinTech College Students Challenge — International Track

Topic E — SME Finance Copilot · v1.0 · October 2026

This report documents LedgerPilot's permission-tier implementation and a structured known-risk list with mitigations and residual-risk ratings. It is a companion to the *Technical Documentation*.

> **Security posture in one line:** LedgerPilot can analyse freely but cannot act without a human saying yes — and every decision it makes is logged.

1. Threat model

As a finance agent, LedgerPilot touches sensitive business data and can, in production, trigger money-adjacent actions (reminders, follow-ups, and — on the roadmap — payments). The assets, actors, and primary threats we design against:

Assets	Threats we design against
Business financial data; the ability to take actions; the integrity of the audit trail.	(a) the agent taking an unintended or unauthorised action; (b) prompt-injection or a bad plan escalating privilege; (c) unexplainable or non-compliant financial recommendations; (d) silent failure with no traceability; (e) data over-exposure.

2. Permission-tier implementation

Access is governed by three ascending tiers. Every tool statically declares the minimum tier it requires via a `REQUIRED_TIER` attribute; the agent runs under a configurable `max_tier` ceiling.

Tier	Name	Side effects	Tools	Enforcement rule
T0	Read-only analysis	None	`analyze_cashflow`, `financial_analysis`	Always permitted.
T1	Recommend / draft	None	`match_financing`	Permitted up to the ceiling.
T2	State-changing action	Yes	`schedule_action`	Permitted up to the ceiling **and** requires an explicit, matching confirmation token — every time.

2.1 The enforcement point

All gating flows through a single `PermissionContext.check()` call, so there is one place to audit and no tool can bypass it:

```
def check(self, required, action_summary=""):
    if required > self.max_tier:
        raise PermissionError_(...)          # ceiling exceeded -> hard denial
    if required == Tier.T2_ACT:
        if self.confirmed_action != action_summary:
            raise ConfirmationRequired(...)   # action not yet confirmed
```

Two independent conditions must both hold for an action to run: the ceiling must allow it, *and* the user must have confirmed the **exact** pending action (matched by its summary string). The agent never sets its own confirmation token — only a user turn does, through the agent loop.

2.2 Operating modes

- **Guarded autonomy (default live mode)** — ceiling T2. The agent may take actions, but each one stops for confirmation. This is the shipped default.
- **Read-only lockdown** — ceiling T0. The agent can analyse but cannot recommend or act; even a user "yes" cannot force an action through. Used for untrusted contexts or compliance-restricted deployments.

2.3 Verification

The permission model is covered by dedicated tests and demonstrated live in `run_scenario.py`. Three outcomes are proven:

Scenario	Expected	Result
Action requested, not yet confirmed	Gated (`ConfirmationRequired`)	**PASS — confirmation prompt shown**
Same action, user confirms	Executes, written to actions ledger	**PASS**
Read-only agent, user says "yes"	Denied (`PermissionError`)	**PASS — ceiling blocks it**

The automated suite additionally asserts: read always allowed; recommend blocked under a read ceiling; action blocked without confirmation; action runs only with a *matching* confirmation; and the ceiling is enforced even when a (stale) confirmation token is present.

3. Auditability

Every turn produces an append-only JSONL trail via `AuditLogger`. Logged event types: `user_message`, `plan`, `tool_call` (with the required tier), `tool_result`, `confirmation_required`, `permission_denied`, `tool_error`, and `assistant_reply`. Authorised state-changing actions are additionally recorded in a separate actions ledger. This gives a complete, replayable record of what the agent saw, decided, and did — the basis for incident review and compliance evidence.

4. Known-risk list

Risks are rated by residual severity *after* the stated mitigation.

#	Risk	Mitigation in place	Residual
R1	Agent takes an unintended state-changing action.	T2 confirmation-always; the agent cannot self-approve; exact-match on the action summary prevents confirming a different action than the one shown.	**Low**
R2	Prompt injection / adversarial input steers the plan toward an action.	Injection can at most *propose* a T2 action; execution still needs a human confirmation and is bounded by the ceiling. Read-only lockdown removes the surface entirely for untrusted contexts.	**Medium**
R3	Unexplainable or non-compliant financial recommendation.	All recommendations are rule-based and return their reasons; the health score exposes its components and weights; every financing reply carries a "not a credit offer / not regulated advice" disclaimer.	**Low**
R4	Silent failure or untraceable behaviour.	Full audit log of every decision and error; deterministic offline mode makes runs reproducible for review.	**Low**
R5	Over-exposure of sensitive financial data.	Datastore is read-only and scoped to the selected business per request; no cross-business data is mixed; no data leaves the process in offline mode.	**Medium**
R6	Hallucinated numbers from the LLM backend.	All figures originate from deterministic tools, not the model; the narrator is instructed to never invent numbers beyond tool results. Offline mode is fully deterministic.	**Low**
R7	Secret/key leakage (live mode).	Keys are read only from environment / `.env` (git-ignored); never logged; never embedded in code or sent to any endpoint other than the configured LLM provider.	**Low**
R8	Stale or incorrect financial advice from outdated data.	Analyses are timestamped and bounded to the six-month window; production roadmap uses live accounting/bank connectors with freshness checks.	**Medium**

5. Compliance stance

- **Not regulated financial advice.** LedgerPilot provides explainable guidance and analysis, not a credit decision or investment advice. Every financing response states this explicitly.
- **Human-in-the-loop for actions.** No money-adjacent action occurs without an explicit, logged user confirmation.
- **Data minimisation.** Only the selected business's data is read for a given request; nothing is shared externally in offline mode.
- **Transparency.** Scores and recommendations ship with their underlying reasons so an owner (or an auditor) can see why.

6. Residual-risk summary

After mitigations, no risk is rated high. The three medium residual risks (R2 prompt injection, R5 data exposure, R8 data freshness) are the priorities for the next iteration: an input-sanitisation / policy layer in front of the planner, per-field data-access scoping, and live connectors with freshness guarantees.
